OP-TEE 주요 프로세스 상세 분석

1. 부팅 프로세스 (Boot Process)

1.1 부팅 시퀀스 개요

OP-TEE의 부팅 프로세스는 다음과 같은 단계로 진행됩니다:

```
1. 하드웨어 초기화
2. 메모리 설정 및 MMU 초기화
3. 스택 카나리 설정
4. 동적 공유 메모리 발견
5. 암호화 서브시스템 초기화
6. 스레드 시스템 초기화
7. TA 로딩 및 초기화
```

1.2 메모리 발견 프로세스 (`discover_nsec_memory`)

```
void  discover_nsec_memory(void)
{
struct core_mmu_phys_mem *mem;
size_t nelems;
void *fdt = get_external_dt();
```

```
// 1. 외부 Device Tree에서 메모리 정보 탐색
if (fdt) {
mem = get_nsec_memory(fdt, &nelems);
if (mem) {
core_mmu_set_discovered_nsec_ddr(mem, nelems);
return;
}
}
// 2. 내장 Device Tree에서 메모리 정보 탐색
fdt = get_embedded_dt();
if (fdt) {
mem = get_nsec_memory(fdt, &nelems);
if (mem) {
core_mmu_set_discovered_nsec_ddr(mem, nelems);
return;
}
}
// 3. 정적으로 등록된 메모리 사용
// 플랫폼별 메모리 설정 활용
}
```

주요 기능:

- **Device Tree 파싱**: 하드웨어 메모리 레이아웃 파악

- **Non-secure 메모리 식별**: 동적 공유 메모리로 사용할 영역 탐색
- **메모리 타입 분류**: `MEM_AREA_DDR_OVERALL` 타입으로 분류

1.3 스택 보호 초기화

```
void  plat_get_random_stack_canaries(void *buf, size_t  ncan, size_t  size)
{
TEE_Result ret;
// 가상화 환경에서는 고정값 사용 (보안 경고)
if (IS_ENABLED(CFG_NS_VIRTUALIZATION)) {
IMSG("WARNING: Using fixed value for stack canary");
memset(buf, 0xab, ncan * size);
goto out;
}
// 하드웨어 RNG를 통한 랜덤 카나리 생성
ret = crypto_rng_read(buf, ncan * size);
if (ret != TEE_SUCCESS)
panic("Failed to generate random stack canary");
out:
// 문자열 기반 공격 방지를 위한 null 바이트 삽입
for (i = 0; i < ncan; i++)
*((uint8_t *)buf + size * i) = 0;
}
```

2. 스레드 관리 시스템 (Thread Management)

2.1 스레드 아키텍처

OP-TEE는 다음과 같은 스레드 구조를 사용합니다:

```
// 스레드 컨텍스트 구조
struct thread_ctx {
struct thread_ctx_regs regs; // CPU 레지스터 상태
enum thread_state state; // 스레드 상태
vaddr_t stack_va_end; // 스택 끝 주소
uint32_t flags; // 스레드 플래그
struct mobj *mobj; // 메모리 객체
// ... 기타 필드들
};

// 코어별 로컬 데이터
struct thread_core_local {
uint32_t curr_thread; // 현재 실행 중인 스레드 ID
uint32_t flags; // 코어 플래그
vaddr_t tmp_stack_va_end; // 임시 스택 주소
int16_t curr_cpu; // 현재 CPU ID
// ... 기타 필드들
};
```

2.2 스택 관리

```
// 스택 구조 (낮은 주소 → 높은 주소)
// [CANARY/2 | STACK_CHECK_EXTRA | STACK_SIZE | CANARY/2]

#define DECLARE_STACK(name, num_stacks, stack_size, linkage) \
linkage uint32_t name[num_stacks] \
[ROUNDUP(stack_size + STACK_CANARY_SIZE + STACK_CHECK_EXTRA, \
STACK_ALIGNMENT) / sizeof(uint32_t)] \
__attribute__((section(".nozi_stack." # name), \
aligned(STACK_ALIGNMENT)))

// 스택 타입별 선언
DECLARE_STACK(stack_tmp, CFG_TEE_CORE_NB_CORE, STACK_TMP_SIZE, /* global */);
DECLARE_STACK(stack_abt, CFG_TEE_CORE_NB_CORE, STACK_ABT_SIZE, static);
DECLARE_STACK(stack_thread, CFG_NUM_THREADS, STACK_THREAD_SIZE, static);
```

스택 보호 메커니즘:

- **스택 카나리**: 스택 오버플로우 탐지
- **정렬 요구사항**: 메모리 정렬을 통한 성능 최적화
- **섹션 분리**: 스택을 별도 메모리 섹션에 배치

2.3 스레드 상태 관리

```
enum thread_state {
THREAD_STATE_FREE, // 사용 가능한 스레드
THREAD_STATE_SUSPENDED, // 일시 중단된 스레드
THREAD_STATE_ACTIVE, // 활성 스레드
};

// 스레드 상태 전환 함수들
static void thread_set_state(uint32_t thread_id, enum thread_state state);
static enum thread_state thread_get_state(uint32_t thread_id);
```

3. TA (Trusted Application) 관리 시스템

3.1 TA 컨텍스트 구조

```
struct tee_ta_ctx {
TEE_UUID uuid; // TA 고유 식별자
uint32_t flags; // TA 플래그 (SINGLE_INSTANCE, CONCURRENT 등)
struct ts_ctx ts_ctx; // 기본 TS 컨텍스트
TAILQ_ENTRY(tee_ta_ctx) link; // 연결 리스트 링크
struct tee_ta_session_head open_sessions; // 열린 세션들
struct mutex mutex; // TA별 뮤텍스
struct condvar cv; // 조건 변수
bool busy; // 사용 중 플래그
```

```
// ... 기타 필드들
};
```

3.2 TA 플래그 시스템

```
// TA 동작 모드 플래그
#define TA_FLAG_SINGLE_INSTANCE (1 << 0) // 단일 인스턴스만 허용
#define TA_FLAG_MULTI_SESSION (1 << 1) // 다중 세션 지원
#define TA_FLAG_INSTANCE_KEEP_ALIVE (1 << 2) // 인스턴스 유지
#define TA_FLAG_SECURE_DATA_PATH (1 << 3) // 보안 데이터 경로
#define TA_FLAG_CONCURRENT (1 << 4) // 동시 실행 허용
#define TA_FLAG_DEVICE_ENUM (1 << 5) // 디바이스 열거 권한
```

3.3 단일 인스턴스 TA 관리

```
#ifndef CFG_CONCURRENT_SINGLE_INSTANCE_TA
static struct condvar tee_ta_cv = CONDVAR_INITIALIZER;
static short int tee_ta_single_instance_thread = THREAD_ID_INVALID;
static size_t tee_ta_single_instance_count;

static void lock_single_instance(void)
{
// tee_ta_mutex가 이미 잠겨있어야 함
```

```
if (tee_ta_single_instance_thread != thread_get_id()) {
// 단일 인스턴스 락이 사용 가능할 때까지 대기
while (tee_ta_single_instance_thread != THREAD_ID_INVALID)
condvar_wait(&tee_ta_cv, &tee_ta_mutex);
tee_ta_single_instance_thread = thread_get_id();
assert(tee_ta_single_instance_count == 0);
}
tee_ta_single_instance_count++;
}

static void unlock_single_instance(void)
{
assert(tee_ta_single_instance_thread == thread_get_id());
assert(tee_ta_single_instance_count > 0);
tee_ta_single_instance_count--;
if (tee_ta_single_instance_count == 0) {
tee_ta_single_instance_thread = THREAD_ID_INVALID;
condvar_signal(&tee_ta_cv);
}
}
#endif
```

3.4 TA 세션 관리

```
struct tee_ta_session {
struct ts_session ts_sess; // 기본 TS 세션
TAILQ_ENTRY(tee_ta_session) link; // 세션 리스트 링크
struct tee_ta_ctx *ctx; // 연결된 TA 컨텍스트
TEE_Identity clnt_id; // 클라이언트 식별자
bool cancel_mask; // 취소 마스크
TEE_Time cancel_time; // 취소 시간
// ... 기타 필드들
};


// 세션 상태 관리
static  bool  tee_ta_try_set_busy(struct tee_ta_ctx *ctx)
{
bool rc = true;
// 동시 실행 가능한 TA는 항상 true 반환
if (ctx->flags & TA_FLAG_CONCURRENT)
return  true;
mutex_lock(&tee_ta_mutex);
// 단일 인스턴스 TA 처리
if (ctx->flags & TA_FLAG_SINGLE_INSTANCE)
lock_single_instance();
if (has_single_instance_lock()) {
```

```
if (ctx->busy) {

// 데드락 방지를 위해 락 해제 후 false 반환

rc = false;

if (ctx->flags & TA_FLAG_SINGLE_INSTANCE)

unlock_single_instance();

}

}

// ... 추가 로직

return rc;

}
```

4. 메모리 관리 시스템 (Memory Management)

4.1 메모리 영역 분류

```
enum teecore_memtypes {

MEM_AREA_NOTYPE, // 타입 없음

MEM_AREA_TEE_RAM, // TEE RAM 영역

MEM_AREA_TEE_COHERENT, // TEE 일관성 메모리

MEM_AREA_TA_RAM, // TA RAM 영역

MEM_AREA_NSEC_SHM, // Non-secure 공유 메모리

MEM_AREA_DDR_OVERALL, // 전체 DDR 메모리

MEM_AREA_TEE_RAM_RX, // TEE 실행 가능 메모리
```

```
MEM_AREA_TEE_RAM_RO, // TEE 읽기 전용 메모리

MEM_AREA_TEE_RAM_RW, // TEE 읽기/쓰기 메모리

// ... 기타 타입들

};
```

4.2 MMU (Memory Management Unit) 관리

```
// MMU 물리 메모리 구조체

struct core_mmu_phys_mem {

enum teecore_memtypes type; // 메모리 타입

paddr_t addr; // 물리 주소

size_t size; // 크기

};


// MMU 설정 함수들

void  core_mmu_set_discovered_nsec_ddr(struct core_mmu_phys_mem *mem,

size_t  nelems);

void  core_mmu_get_mem_by_type(enum teecore_memtypes type,

vaddr_t *s, vaddr_t *e);
```

4.3 페이지 할당자

```
// 페이지 할당 인터페이스
```

```
struct page_alloc_ctx {
struct mutex mutex; // 할당자 뮤텍스
void *pool; // 메모리 풀
size_t pool_size; // 풀 크기
// ... 기타 필드들
};

// 페이지 할당/해제 함수들
void *page_alloc(size_t  size, enum page_alloc_type type);
void  page_free(void *ptr, size_t  size);
```

5. 암호화 서브시스템 (Cryptographic Subsystem)

5.1 암호화 드라이버 아키텍처

```
// 암호화 컨텍스트 구조
struct crypto_ctx {
const  struct crypto_ops *ops; // 암호화 연산 함수 포인터
void *ctx; // 구현별 컨텍스트
};

// 암호화 연산 인터페이스
struct crypto_ops {
```

```
TEE_Result (*init)(struct crypto_ctx *ctx, ...);

TEE_Result (*update)(struct crypto_ctx *ctx, ...);

TEE_Result (*final)(struct crypto_ctx *ctx, ...);

void (*free_ctx)(struct crypto_ctx *ctx);

};
```

5.2 하드웨어 RNG 지원

```
// RNG 드라이버 인터페이스

struct rng_ops {

TEE_Result (*read)(void *buf, size_t len);

void (*add_entropy)(const  void *buf, size_t len);

};


// 플랫폼별 RNG 구현 예시

TEE_Result crypto_rng_read(void *buf, size_t  blen)

{

if (rng_ops && rng_ops->read)

return  rng_ops->read(buf, blen);

// 소프트웨어 폴백

return  rng_fortuna_read(buf, blen);

}
```

5.3 키 관리 시스템

```
// 키 객체 구조
struct tee_cryp_obj {
TEE_ObjectInfo info; // 객체 정보
TEE_Attribute *attrs; // 속성 배열
uint32_t attr_count; // 속성 개수
void *key; // 실제 키 데이터
size_t key_size; // 키 크기
};

// 키 파생 함수 (HKDF, PBKDF2 등)
TEE_Result tee_cryp_hkdf(uint32_t  hash_id, const  uint8_t *ikm, size_t  ikm_len
const  uint8_t *salt, size_t  salt_len,
const  uint8_t *info, size_t  info_len,
uint8_t *okm, size_t  okm_len);
```

6. 인터럽트 및 예외 처리

6.1 인터럽트 처리 아키텍처

```
// 인터럽트 핸들러 구조
struct itr_handler {
```

```
itr_handler_t handler; // 핸들러 함수

uint32_t it; // 인터럽트 번호

uint32_t flags; // 플래그

void *data; // 핸들러 데이터

};


// 인터럽트 등록/해제

void  itr_add(struct itr_handler *handler);

void  itr_enable(uint32_t  it);

void  itr_disable(uint32_t  it);
```

6.2 예외 처리 메커니즘

```
// 예외 타입 정의

enum fault_type {

FAULT_TYPE_UNDEF_ABORT, // 정의되지 않은 명령어

FAULT_TYPE_PREFETCH_ABORT, // 프리페치 중단

FAULT_TYPE_DATA_ABORT, // 데이터 중단

FAULT_TYPE_ALIGNMENT, // 정렬 오류

};


// 예외 핸들러 인터페이스

bool  abort_handler(uint32_t  abort_type, vaddr_t  fault_addr,
```

```
struct thread_abort_regs *regs);
```

7. 시스템 콜 인터페이스

7.1 시스템 콜 디스패치

```
// 시스템 콜 번호 정의
#define SYSCALL_SYS_RETURN 0
#define SYSCALL_SYS_LOG 1
#define SYSCALL_SYS_PANIC 2
#define SYSCALL_SYS_GET_TIME 3
// ... 기타 시스템 콜들

// 시스템 콜 핸들러 테이블
static const syscall_t syscall_table[] = {
[SYSCALL_SYS_RETURN] = syscall_sys_return,
[SYSCALL_SYS_LOG] = syscall_sys_log,
[SYSCALL_SYS_PANIC] = syscall_sys_panic,
// ... 기타 핸들러들
};
```

7.2 매개변수 검증

```
// 사용자 공간 포인터 검증
```

```
TEE_Result copy_from_user(void *kaddr, const  void *uaddr, size_t  len)

{

if (!tee_mmu_is_vbuf_inside_um_private(uaddr, len))

return TEE_ERROR_ACCESS_DENIED;

memcpy(kaddr, uaddr, len);

return TEE_SUCCESS;

}



// 버퍼 경계 검사

bool  tee_mmu_is_vbuf_inside_um_private(const  void *va, size_t  size)

{

return  core_mmu_user_va_range_is_defined(va, size) &&

core_mmu_user_mapping_is_active();

}
```

이 문서는 OP-TEE의 주요 프로세스들에 대한 상세한 분석을 제공합니다. 각 시스템은 보안성, 성능, 확장성을 고려하여 설계되었으며, ARM TrustZone의 하드웨어 보안 기능을 최대한 활용하도록 구현되어 있습니다.